

Operations in x86-64 Assembly

Assembly ~~prog~~ source code →

- 1) Assembler directive
- 2) Opcodes
- 3) Labels.

```
segment .data
a dq 175
b dq 4097
```

→ Assembler directive

dq → . q → quad word

b → byte

w → word (2 bytes)

q → quad word (8 bytes)

d → double word (4 bytes)

segment .text →

→ assembler
directive

global main →

→ assembler
directive

```
main: mov rax, [a]
      add rax, [b]
      xor rax, rax
      ret
```

int a = 175

Labels are memory locations that refer to some part of the code. They need to be defined within segment .text.

Inline functions: → Copy the code instead of jumping it.

mov rax, [a] → Copy the ~~of~~ content of a into the register rax

add rax, [b] → Add the content of b into rax and store the result in rax.

64-bit registers

16-bit registers 32-bit registers

rax	→ accumulator	ax	eax
rbx	→ base register	bx	ebx
rcx	→ count register	cx	ecx
rdx	→ data register	dx	edx
rsi	→ source index	si	esi
rdi	→ destination index	di	edi
rbp	→ base pointer	bp	ebp
rsp	→ stack pointer	sp	esp
r8			
r9			
⋮			
r15			

ax, eax and rax are the same physical register.

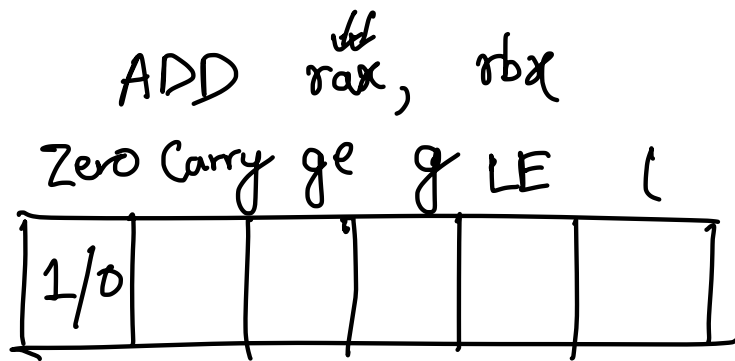
Register Naming
Mechanism in
x86-64 Processors

Loops: → Unconditional jump

JMP main

for (i=0; i < 5; i++) {
 line 1; line 2; line 3
}

Program Status Word (PSW) stores some flags based on the last program operation.



CMP rax, rbx

PSW $\rightarrow$ Special purpose register That cannot be set by a direct instruction.

JZ main

JNZ main

JC JGE JLE

JNC JG JL

If conditions must be done using two instructions.

MOV i, #0 CMP i, 5

LOOP:-

line 1

line 2

line 3

..

$\rightarrow$ Body of the loop

ADD i, 1

CMP i, 5

JL LOOP

XOR rax, rax

ret

Exercise: Complete this program with proper assembler directives and check whether you are able to run the loop.

Addressing Modes in Assembly

[MOV] $\rightarrow$ Intel machines use only this for all addressing modes!

MVI $\rightarrow$ Move Immediate

MOV rax, a $\rightarrow$ Direct addressing

In ARM,
you need LD
instruction for this.

[a]

Indirect addressing
(Interpret the variable's data
as an address, and
bring the data stored
in the address
into the register)

MOV a, rax

MOV [rax], a $\rightarrow$ In ARM, you need
ST instruction.

Opcode does not depend on the operand

$\Rightarrow$ orthogonal
Intel supports orthogonality to make assembly
programming simpler.

$\rightarrow$ Processor architecture and assembler both
become more complex.

1) Vector operations

→ Specialized instructions so that multiple operations of same type can be done using a single instruction.

```
for (int i=0; i<10; i++) {  
    A[i] = A[i] + B[i];  
}
```

$A = A + B$
ADDV A, B

→ Additional instructions added to handle multimedia and machine learning workloads.

2) Floating point → Registers and instructions are different.

```
float a, b, c;
```

```
a = b + c;
```

```
int a, b, c;
```

```
a = b + c;
```

No orthogonality of operations with respect to data types

3) Embed assembly programs within C language without making it a distinct function.

```
a = b + c;  
--asm-- add ....  
printf( " - - " );
```
